

Action Model Learning with Guarantees: Theoretical Framework and Heuristic Optimization

Claudio Metelli and Mattia Cherubini

A.Y. 2024/2025

Abstract

In this work, we present a theoretical analysis and an enhanced implementation of the Action Model Learning algorithm based on Version Spaces (Aineto and Scala 2024). This framework interprets learning as a search within a hypothesis space, maintaining strict bounds to guarantee *soundness* and *completeness*. Furthermore, to address the convergence in random exploration, we introduce a greedy heuristic strategy for optimized demonstration generation.

1 Introduction

Action model engineering, which is fundamental to model-based reasoning and automatic planning in Artificial Intelligence, has historically been one of the main bottlenecks due to its complexity and propensity for error. *Action Model Learning* (AML) approach addresses this problem by automating the acquisition of domain dynamics, computing an approximation of them from demonstrations provided by the agent.

This work is based on the learning algorithm that embodies the theory of AML based on *Version Spaces* (Mitchell 1982). This approach interprets the learning task as a search for all hypotheses consistent with the training data, i.e., the observed transitions from state s to state s' through action a : $\langle s, a, s' \rangle$. The algorithm dynamically maintains a compact representation of the entire set of action models that could describe the agent's behavior.

The main utility of this methodology lies in its ability to construct a rigorous and formally grounded model. Direct analysis of transitions allows the preconditions and effects of actions to be inferred, enabling the observer to deduce directly and formally what the agent is capable of doing. Furthermore, thanks to its theoretical derivation, the framework is intrinsically oriented towards providing formal guarantees:

Sound Model : A model $\mathcal{M}$ is defined *sound* w.r.t. the true model $\mathcal{A}$ if every valid transition in $\mathcal{M}$ is guaranteed to be valid in $\mathcal{A}$. It represents a “safe” approximation, ideal for scenarios where execution correctness is the

highest priority.

Complete Model : A model $\mathcal{M}$ is defined *complete* if every valid transition in $\mathcal{A}$ is included in $\mathcal{M}$. It represents an optimistic view, useful for recognizing the existence of potential plans.

This feature makes it an ideal element to be paired with more powerful but less accurate models, such as *Large Language Models* (LLMs), as already shown by Katz et al. 2024. In a hybrid context, the rigorous model could act as a formal verification mechanism for the actions suggested by the LLM, increasing the overall reliability and transparency of the system.

2 Theoretical Framework

The *Action Model Learning* (AML) problem can be formalized within the context of classical planning. Our goal is to infer the dynamics of a domain by observing an agent's interactions with the environment.

2.1 Action Model and Transitions

An Action Model is formally defined as a tuple $M = \langle F, A, pre, eff \rangle$, where:

- $F = \{f_1, \dots, f_n\}$ is a finite set of Boolean state variables, called fluents. A state s is defined as a subset of literals L derived from F .
- A is a finite set of actions.

- $pre : A \rightarrow 2^L$ defines the preconditions of an action, i.e., the literals that must be true for the action to be executable.
- $eff : A \rightarrow 2^L$ defines the effects of an action, i.e., how the state of the world changes after execution.

The dynamics of the system are governed by two fundamental rules: **applicability** and **state transition**. An action $a \in A$ is considered applicable in a state s if and only if its preconditions are satisfied in the current state, i.e., $pre(a) \subseteq s$. If action a is applicable in s , execution produces a new state s' calculated according to the following transition function:

$$s' = (s \setminus \overline{eff(a)}) \cup Eff(a)$$

where $\overline{eff(a)}$ represents negative effects (fluents that become false) and $eff(a)$ represents positive effects (fluents that become true).

2.2 The Learning Problem

The input for the learning problem consists of a set of demonstrations D . A demonstration represents a trace of the execution (or attempted execution) of an action. We distinguish between two types of demonstrations:

- **Positive Demonstration:** triple $d = \langle s, a, s' \rangle$ indicating that action a was successfully applied in s , producing state s' .
- **Negative Demonstration:** triple $d = \langle s, a, \perp \rangle$ indicating that the application of action a in state s failed (indicated by the symbol $\perp$).

An **Action Model Learning problem** is therefore defined by the tuple $\Lambda = \langle F, A, D \rangle$. The goal is to find a model M that is consistent with the set of proofs D , i.e., a model that satisfies the following conditions:

- For each positive demonstration $\langle s, a, s' \rangle \in D$, the model must ensure that the action is applicable ($pre(a) \subseteq s$) and that the resulting state corresponds to the observed one ($s' = (s \setminus \overline{eff(a)}) \cup eff(a)$).
- For every negative proof $\langle s, a, \perp \rangle \in D$, the model must guarantee that the action is not applicable in state s ($pre(a) \not\subseteq s$).

The set of all models that satisfy these properties is denoted as $\mathcal{M}_D$, the space of consistent models. The learning problem is decomposed into learning preconditions ($Pre(a)$) and effects ($Eff(a)$) separately for each action $a \in A$ of the problem (where A is the set of all possible actions). Since explicitly enumerating all consistent hypotheses is computationally intractable for PDDL domains, the framework maintains the Version Space $\mathcal{V}$ compactly via its boundaries: the *Lower Bound* ($\mathcal{L}$) and the *Upper Bound* ($\mathcal{U}$). The true model $\mathcal{A}$ is guaranteed to lie within these boundaries:

$$\mathcal{L} \subseteq \mathcal{A} \subseteq \mathcal{U}$$

2.3 Hypothesis Spaces and Ordering

Since the space of all possible models is finite but exponential, explicitly enumerating the solutions is intractable. We therefore adopt the **Version Spaces** framework to represent the set of all consistent models in a compact way.

We define $\mathcal{H}$ as the Hypothesis space for a given function (precondition or effect). The Version Space $\mathcal{V}_{\mathcal{H},D}$ is defined as the subset of $\mathcal{H}$ containing all hypotheses h that are consistent with the set of proofs D .

Instead of listing the hypotheses, $\mathcal{V}_{\mathcal{H},D}$ is represented by its boundaries with respect to a partial order relation $\leq$:

- **Lower Bound ($\mathcal{L}$):** The boundary of the most specific (or restrictive) hypotheses.
- **Upper Bound ($\mathcal{U}$):** The boundary of the most general (or permissive) hypotheses.

A hypothesis h belongs to the Version Space if and only if it lies between these boundaries:

$$\mathcal{V}_{\mathcal{H},D} = \{h \in \mathcal{H} \mid \exists (h_{\mathcal{L}}, h_{\mathcal{U}}) \in \mathcal{L} \times \mathcal{U} : h_{\mathcal{L}} \leq h \leq h_{\mathcal{U}}\}$$

To define the boundaries, we must establish a partial ordering for the hypothesis spaces of preconditions and effects.

Precondition Space ($\mathcal{H}_p$) A hypothesis $h \in \mathcal{H}_p$ represents a set of literals required for action applicability. The partial order $\leq$ is defined following the idea that hypothesis h_1 more restrictive than another hypothesis h_2 if h_1 includes all the literals of h_2 :

$$h_1 \leq h_2 \iff h_1 \supseteq h_2.$$

Consequently:

- The Lower Bound of preconditions $\mathcal{L}_p$ is initialized to the set of all possible literals L , since it is the most restrictive, and the action should match the worst possible scenario.
- The Upper Bound of preconditions $\mathcal{U}_p$ is initialized to $\emptyset$, since it is the most permissive and the action should always be applicable.

Effect Space ($\mathcal{H}_e$) A hypothesis $h \in \mathcal{H}_e$ represents the state changes induced by the action a . The partial order $\leq$ is defined following the idea that hypothesis h_1 more restrictive than another hypothesis h_2 if every literal in h_1 is included in h_2 :

$$h_1 \leq h_2 \iff h_1 \subseteq h_2.$$

Consequently:

- The Lower Bound of effects $\mathcal{L}_e$ is initialized to $\emptyset$, since no effects is guaranteed to be possible.
- The Upper Bound of effects $\mathcal{U}_e$ is initialized to L , since the most optimistic view is that all effects are possible.

2.4 Update Rules

The evolution of the Version Space is driven by algebraic set operations applied to the bounds upon observing a demonstration d . The original paper uses more complex operations as a bug check but, as demonstrated in the same paper, they can be simplified. We report how to squeeze or enlarge the upper and lower bounds of the domain using insiemistic operations.

Positive Demonstrations When an action is successfully applied, the model must allow it and predict the outcome.

- **Lower Bound of Preconditions ($\mathcal{L}_p$):** The bound is updated via intersection; the main idea of the lower bound of preconditions is that if a literal is not present in s , it cannot be a necessary precondition.

$$\mathcal{L}_p \leftarrow \mathcal{L}_p \cap s$$

This relaxes the bound, removing unnecessary literals from the set of necessary conditions.

- **Lower Bound of Effects ($\mathcal{L}_e$):** The bound accumulates observed changes from s to s' using union operation of the difference between the two states.

$$\mathcal{L}_e \leftarrow \mathcal{L}_e \cup (s' \setminus s)$$

- **Upper Bound of Effects ($\mathcal{U}_e$):** The bound restricts possible effects via intersection with the resulting state. Following the same idea of the lower bound of preconditions, if a literal is not present in s' , it cannot be an effect of the action.

$$\mathcal{U}_e \leftarrow \mathcal{U}_e \cap s'$$

Negative Demonstrations When an action fails, the model must not enable it.

- **Upper Bound of Preconditions ($\mathcal{U}_p$):** If a hypothesis $h \in \mathcal{U}_p$ is a subset of s (meaning it would incorrectly allow the action), it must be specialized. The algorithm generates new candidate hypotheses by adding literals from $\mathcal{L}_p \setminus s$ to h , effectively creating minimal specializations that exclude the failing state s .

2.5 Model Extraction

After processing all demonstrations from the demonstration set, the sound model and the complete model can be derived. These model are expressed through PDDL language.

Sound Model The Sound Model is designed to be restrictive and reliable. It is constructed exclusively using the two Lower Bounds ($\mathcal{L}_p$ and $\mathcal{L}_e$), which by definition represent certain and necessary facts.

- **Preconditions** are extracted from the literals present in the lower bound set of each action. Since this set is the result of the intersection of all initial states in positive transitions, we are certain that it contains all the real preconditions for an action. Each symbolic literal is formalized as a PDDL predicate and added as a precondition.
- **Effects** are extracted from the lower bound set, which was constructed as the union of all observed effects. This set represents the facts that the action has certainly caused at least once. These literals are formalized as a PDDL predicate and added as effects of the action in the new domain.

Complete Model The Complete Model is designed to be more permissive and general, even at the cost of including potentially incorrect literals. It is constructed exclusively using the two Upper Bounds ($\mathcal{U}_p$ and $\mathcal{U}_e$).

- **Preconditions:** the upper bound of preconditions is a set of hypothesis, each of which is considered a valid alternative in the model, so that even if only one of them is satisfied, the action is considered applicable. This is translated into the PDDL model preconditions of an action through the use of the OR statement, which combines all possible hypotheses in $\mathcal{U}_p$, verifying that even just one of them is verified. This implies that the action is applicable if at least one of these hypothesis is satisfied.
- **Effects:** in this case, we implement a logic of action decomposition from the original action. The hypothesis space of effects is calculated, covering all possible literals between the two bounds. The original action in the domain is replaced by a set of derived actions and each one of these actions a_i is assigned a discrete and deterministic set of effects that correspond to a single hypothesis in the hypothesis space of effects. This method makes the uncertainty of the effects explicit through a disjunction at the action level. At the same time, the model is “angelic”: using multiple actions to describe multiple effects, we can choose the outcome effect of an action in an arbitrary way.

As highlighted by the intrinsic characteristics of soundness and completeness, the Sound Model is often considered to be of greater relevance in terms of predictive reliability with regard to inferred effects, thanks to its basis of certain facts, while Complete Model can be significantly faster in reaching the goal, but without any assurance that the plan is actually feasible.

3 Demonstration Generation

As established in the theoretical analysis, the algorithm processes the entire set of provided demonstrations. Convergence occurs when the Sound and Complete models coincide, identifying the true model. However, random exploration of the state space often yields redundant data, slowing down the convergence of the two bounds.

To maximize the effectiveness of the approximation, we shift the focus from refining the learning algorithm to optimizing the selection of demonstrations. We propose a greedy heuristic strategy based on the concept of “*Useful Preconditions*”.

3.1 Useful Preconditions

We define the set of *Useful Preconditions* $\mathcal{UP}$ for a given action a as the set of all possible literals that are not part of the action’s formal preconditions in the true model.

$$\mathcal{UP}(a) = L \setminus \text{Pre}(a)$$

If the generation process randomly visits states where certain non-required literals are consistently true, the intersection operation used to squeeze the lower bound of preconditions ($\mathcal{L}_p \cap s$) will erroneously retain them in the Lower Bound.

To counter this, the heuristic aims to maximize the coverage of these literals. By actively seeking to execute action a in states where different subsets of $\mathcal{UP}(a)$ are present, we try to avoid that a non-required literal is accidentally present in all positive demonstrations. This diversity forces the intersection of all observed states to discard these irrelevant literals, accelerating the convergence of $\mathcal{L}_p$ towards the minimal set of true preconditions.

3.2 Heuristic Score and Generation

The generation process starts from a PDDL domain and a PDDL problem (with initial state s_0 , goal g). It utilizes the Unified Planning (Micheli et al. 2025) Sequential Simulator, a simulator that, starting from a state s and an action a , is able to say whether the application of the same action is possible, and if so, what the next state s' is, or if it is not possible, thus returning $\perp$.

For every state s , the algorithm evaluates all applicable actions. For each action a , it simulates the transition to s' and calculates a heuristic score:

$$\text{Score}(a) = |\{l \in \mathcal{UP}(a) \mid l \in s'\}|$$

The action a_{next} that covers the highest number of useful preconditions is selected. This look-ahead ensures that every step maximizes the information gain regarding the action’s constraints. If no new useful preconditions can be explored from s , a standard planner is invoked to advance the state towards the goal g , ensuring the process does not stall, and the action a_{next} is selected. Of course, when a new state is visited, the Useful Preconditions set $\mathcal{UP}$ is updated by removing literals that have been explored in the current step.

Finally, the optimal transition $\langle s, a_{next}, s' \rangle$ is recorded; from the same state s , the algorithm randomly samples actions a_{neg} that are not applicable and triples $\langle s, a_{neg}, \perp \rangle$ are then added to the dataset.

The pseudo-code can be found in the appendix section A.

4 Experimental Evaluation

In this section, we will evaluate the performance of the AML algorithm in order to provide an objective measure of the effectiveness of the Sound and Complete models produced on a selection of planning domains, using appropriate metrics for each property.

We select 4 domains named BLOCKSWORLD, GARDENWORLD, ZENO and GROCERY, that present an increasing level of complexity and challenge the algorithm on different aspects, as suggested by their structure. From these domains, specific PDDL problems were generated. These problems, together with the respective domain, were then submitted to the main process pipeline, as described in Figure 1. This phase allowed the collection of a dataset necessary for training the AML model, the final results of which are analyzed in the following sections.

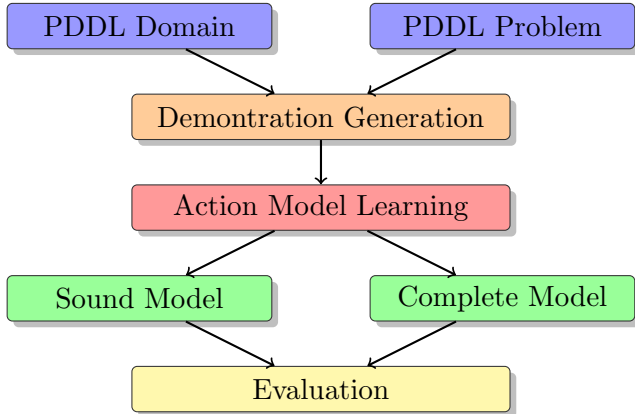


Figure 1: Algorithm pipeline for evaluation of the performances.

4.1 Evaluation Criteria

The evaluation of the learned models is performed by comparing the Sound Model, derived from Lower Bounds $\mathcal{L}_p$ and $\mathcal{L}_e$, and the Complete Model, derived from the Upper Bounds $\mathcal{U}_p$ and $\mathcal{U}_e$, with the original domain. Specifically, metrics that measure structural accuracy are used, i.e. how much the learned preconditions/effects overlap/disjoint with those of the true model. In particular, Precision and Recall metrics are used.

- **Precision** measures the quality of the individual hypotheses generated i.e. the percentage of correctly predicted predicates among

all the predicted predicates. A low precision value suggests that the hypotheses in the set are composed of an excessive number of predicates that do not belong to the real model.

- **Recall** measures the ability of the model to infer the predicates required by the real model i.e. the percentage of correctly predicted predicates among the real predicates. A low recall value indicates that the set of hypotheses does not adequately covers all the predicates necessary for the validity of the model.

Sound Model. In order to evaluate the behaviour of the Sound Model we look at the intersection between the learned preconditions/effects and the real ones.

For preconditions we calculate the precision of learned predicates i.e. the ratio between cardinality of the learned set and the real one. Ideally, a high value confirms that the model is not introducing false positives in an attempt to generalize.

For effects, on the other hand, the metric evaluates whether the lower bound of effects has captured all effects that occurred through the recall metrics. A 100% in this measure indicates that the model has perfectly learned the deterministic effects of the domain.

Complete Model. The Complete Model is evaluated based on its ability to maximize coverage, and therefore completeness, accepting a compromise on precision.

For preconditions we have that precision and recall metrics are both used directly, since the set is a disjunction of hypotheses, that can overestimate or underestimate at the same time.

For effects, the analysis focuses on precision, measuring the ratio between learned effects and real ones. Since the logic of the Complete Model is optimistic, it is expected that the upper bound of effects will be an overestimate of the actual set, accepting false positives and confirming its nature as a potentially unsound model.

4.2 Implicit Correlations and Related Predicates.

A fundamental aspect that emerged from the analysis of the Lower Bound of Preconditions $\mathcal{L}_p$ is the tendency of the algorithm to learn implicit correlations or redundant predicates that are consistently true in the state, but which are not strictly necessary for the logic validity of the PDDL preconditions.

tion. This event is not an error of the algorithm, but reflects the high domain dependency: the algorithm, operating by set operations on the facts of the state, captures everything that is consistently true, even if it is not essential for the logical purposes of the minimal PDDL action. For example, when the algorithm computes the Lower Bound of preconditions through the intersection operations, it includes any fact that is invariably true. These additional facts are not required by the formal definition of the action, but are a consequence of the domain definition and their inclusion makes the Sound Model redundant with respect to the minimal PDDL model, but not logically incorrect.

There are specific cases to prove that, like mutual exclusion, a phenomenon which occurs when two predicates are mutually exclusive by definition of the problem, and one of them is the explicit precondition. The algorithm, by intersecting all states, captures both the action and the implicit fact and the result is that the resulting Sound Model therefore includes this last as a redundant predicate. An example is in the domain BLOCKSWORLD where the **put-down** action explicitly requires (**holding ?x**). In the PDDL state, the truth of (**holding ?x**) implies that (**arm-empty**) is false. So in the Sound Model there will be (**not (arm-empty)**) as redundant predicate.

Another case which is important to include in this discussion concerns domain-invariant dependencies, which occurs when the specific problem imposes an invariance that is not required by the individual action. For example in the domain GROCERY there are predicates the form of (**connected ?a ?b**) that models the connections between compartments and if these are always generated symmetrically, the Lower Bound may include the symmetric form, even though the move action only requires the connection in one direction. This is a learned invariance that comes from the structure of the problem and cannot be avoided, yet another confirm of the caution of the model.

As last observation, we observed that Lower Bound of effects always correspond to the real set of effects. This result is obtained following the definition of state transition reported in Section 2.1. Given the transition induced by an action a in state s , $\langle s, a, s' \rangle$, the literals which are true (resp. false) in state s and become false (resp. true) in state s' are the effect of the action, so the differ-

ence between the states s' and s perfectly captures the effect of the action, and so the algorithm.

4.3 Analysis of ZENO Domain

We choose the ZENO domain to show results of the algorithm in a more detailed way. The application of AML algorithm on domain obtained the results shown in Figure 2, offering significant insights into the behavior of the algorithm.

Sound Model Performance

The Sound Model demonstrates absolute reliability in learning effects. As seen in the graphs, the Lower Bound of effects achieved 100% recall across all actions, like in the rest of the domains. This phenomenon is already explicated in Section 4.2.

Regarding preconditions, the model exhibits high precision but lower recall, due to implicit correlations and related preconditions. For example, we introduced the **zoom** action, which consists in a fly action that uses two levels of fuel instead of just one; the PDDL action is described in Listing 1.

Listing 1: Zoom PDDL Action

```
(:action zoom
:parameters (
  ?a - aircraft
  ?c1 ?c2 - city
  ?l1 ?l2 ?l3 - flevel)
:precondition (and
  (at-aircraft ?a ?c1)
  (route-zoom ?c1 ?c2)
  (fuel-level ?a ?l1)
  (next ?l2 ?l1)
  (next ?l3 ?l2))
:effect (and
  (not (at-aircraft ?a ?c1))
  (at-aircraft ?a ?c2)
  (not (fuel-level ?a ?l1))
  (fuel-level ?a ?l3))
)
```

The sound domain adds 10 negative predicates, including, for example (**not (fuel-level ?a ?l3)**) and (**not (next ?l1 ?l2)**). These type of predicates are implicit correlations: considering (**not (next ?l1 ?l2)**), it is a direct consequence of the real precondition (**next ?l2 ?l1**). These precondition cannot be avoided by the algorithm, and the only solution could be excluding negative preconditions, considering that our domains only utilize positive predicates as preconditions. In this

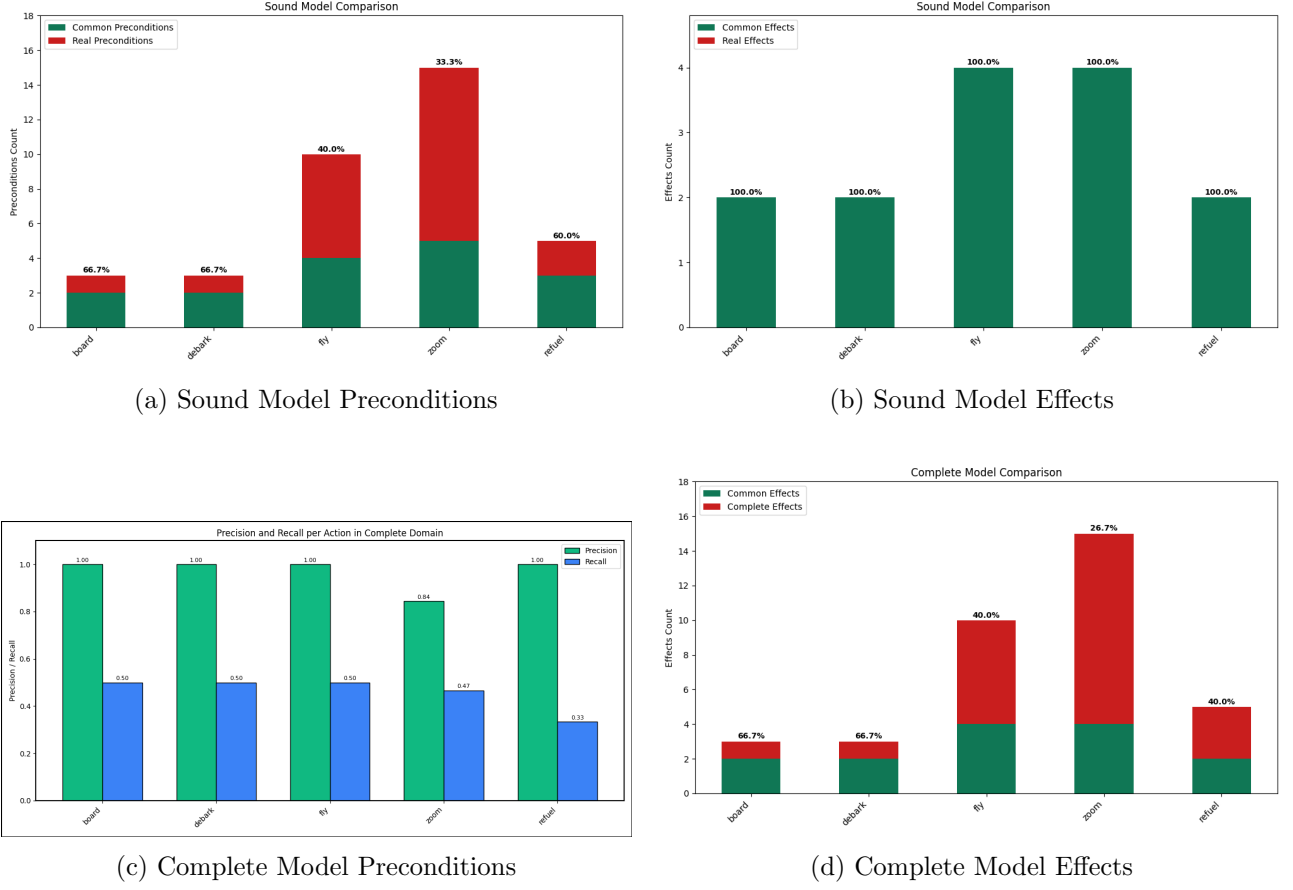


Figure 2: Metrics obtained by the Sound Model and Complete Model in the ZENO domain.

case, we would achieve a 100% recall in each action of the sound domain in ZENO, also considering that our demonstration set is build to squeeze this set.

Complete Model Performance

The Complete Model highlights the challenges of the Upper Bound. The precision of effects drops to approximately to 26.7% for complex actions like **zoom**. This confirms that $\mathcal{U}_e$ remains an overestimate, including false positive effects that were not ruled out by intersection.

An analysis similar to that carried out for the sound preconditions can be made. In fact, we can notice the presence of some implicit correlation and related predicates. Considering the action **fly**, which is represented in the real PDDL domain as described in Listing 2, we can find in its Upper Bound, e.g. the most optimistic hypothesis, predicates such as `(not (route-zoom ?c1 ?c2))` or `(not (fuel-level ?a ?l1))`.

Listing 2: Zoom PDDL Action

```
(:action fly
:parameters (
```

```
?a — aircraft
?c1 ?c2 — city
?l1 ?l2 — flevel)
:precondition (and
(at-aircraft ?a ?c1)
(route-fly ?c1 ?c2)
(fuel-level ?a ?l1)
(next ?l2 ?l1))
:effect (and
(not (at-aircraft ?a ?c1))
(at-aircraft ?a ?c2)
(not (fuel-level ?a ?l1))
(fuel-level ?a ?l2))
)
```

We can immediately notice that if we **fly**, and so we do not **zoom**, from **c1** to **c2**, the route will always be a **route-fly** and not a **route-zoom**, and this is captured by the AML algorithm in an explicit way, adding non necessary predicates in the action effects that are implicit correlations.

For preconditions, while precision of individual hypotheses is high, meaning that almost every precondition that we predict is correct, the overall recall is low, meaning that we are not able to recognize all the necessary precondition. In order to

obtain better recall, the hypotheses in the preconditions Upper Bound should be expanded. This can be done by refining the set of demonstrations through an appropriate heuristic, in order to expand the bound in a more appropriate way. This suggests that the random sampling of negative examples, while helpful, can be improved.

5 Conclusions

In conclusion, we presented a practical way to develop the AML algorithm, showing that significant result as been reached using this algorithm. While the Sound Model proves to be highly reliable for safe planning, the results highlight the inherent difficulty in achieving a tight Complete Model without exhaustive negative evidence.

In fact, we notice that the best way to improve performances does not rely on the algorithm itself, but on the demonstration presented to the algorithm. Following this idea, a simple heuristic method has been presented in order to improve accuracy of the overall algorithm, but more changes can be added: for example, an online algorithm that combines both AML algorithm and exploration of the space following some heuristic in order to collect useful demonstrations could be a significant improvement for the overall accuracy of the model.

More material can be found in the github repository (Metelli and Cherubini 2025).

References

- Aineto, Diego and Enrico Scala (Nov. 2024). “Action Model Learning with Guarantees”. In: *Proceedings of the TwentyFirst International Conference on Principles of Knowledge Representation and Reasoning*. KR-2024. International Joint Conferences on Artificial Intelligence Organization, 801–811. DOI: [10.24963/kr.2024/75](https://doi.org/10.24963/kr.2024/75). URL: <http://dx.doi.org/10.24963/kr.2024/75>.
- Katz, Michael et al. (2024). *Thought of Search: Planning with Language Models Through The Lens of Efficiency*. arXiv: 2404.11833 [cs.AI]. URL: <https://arxiv.org/abs/2404.11833>.
- Metelli, Claudio and Mattia Cherubini (2025). *Action Model Learning with Guarantees*. <https://github.com/claudiometelli/action-model-learning-guarantees>. Robotics Course Project, Università degli Studi di Brescia.
- Micheli, Andrea et al. (2025). “Unified Planning: Modeling, manipulating and solving AI planning problems in Python”. In: *SoftwareX* 29, p. 102012. ISSN: 2352-7110. DOI: <https://doi.org/10.1016/j.softx.2024.102012>. URL: <https://www.sciencedirect.com/science/article/pii/S2352711024003820>.
- Mitchell, Tom M. (1982). “Generalization as search”. In: *Artificial Intelligence* 18.2, pp. 203–226. ISSN: 0004-3702. DOI: [https://doi.org/10.1016/0004-3702\(82\)90040-6](https://doi.org/10.1016/0004-3702(82)90040-6). URL: <https://www.sciencedirect.com/science/article/pii/0004370282900406>.

A Pseudocode for Demonstration Generation

Algorithm 1 Generate Demonstration Set

```
1: FUNCTION GENERATEDEMONSTRATIONSET(Problem, Domain)
2:  $s \leftarrow \text{Problem.InitialState}$ 
3:  $G \leftarrow \text{Problem.Goal}$ 
4:  $\text{DemonstrationSet} \leftarrow \emptyset$ 
5:  $\mathcal{UP} \leftarrow \emptyset$ 
6:  $\mathcal{UP}.\text{Initialize}()$ 
7: while  $\mathcal{UP}$  IS NOT EMPTY AND  $s$  IS NOT  $G$  do
8:    $\text{Scores} \leftarrow \emptyset$ 
9:   for each applicable action  $a$  in  $s$  do
10:     $\mathcal{H}(a) \leftarrow \text{CalculateHeuristicScore}(a, s, \mathcal{UP})$ 
11:     $\text{Scores}[a] \leftarrow \mathcal{H}(a)$ 
12:   end for
13:    $\mathcal{H}_{\max} \leftarrow \max(\text{Scores})$ 
14:    $a_{\text{next}} \leftarrow \text{NULL}$ 
15:   if  $\mathcal{H}_{\max} > 0$  then
16:      $a_{\text{next}} \leftarrow \text{GetActionWithMaxScore}(\text{Scores})$ 
17:   else
18:      $a_{\text{next}} \leftarrow \text{Planner.FindNextAction}(s, G)$ 
19:   end if
20:    $s' \leftarrow \text{Simulator.Apply}(s, a_{\text{next}})$ 
21:    $\mathcal{UP}.\text{RemoveCovered}(a_{\text{next}}, s')$ 
22:    $\text{DemonstrationSet.ADD}(\text{PositiveExample}(s, a_{\text{next}}, s'))$ 
23:    $\text{DemonstrationSet.ADD\_ALL}(\text{NegativeRandomExamples}(s))$ 
24:    $s \leftarrow s'$ 
25: end while
26: if  $s$  IS NOT  $G$  then
27:    $\text{RemainingPlan} \leftarrow \text{Planner.FindPlan}(s, G)$ 
28:   for each action  $a$  in  $\text{RemainingPlan}$  do
29:      $s' \leftarrow \text{Simulator.Apply}(s, a)$ 
30:      $\text{DemonstrationSet.ADD}(\text{PositiveExample}(s, a, s'))$ 
31:      $\text{DemonstrationSet.ADD\_ALL}(\text{NegativeExamples}(s))$ 
32:      $s \leftarrow s'$ 
33:   end for
34: end if
35: RETURN  $\text{DemonstrationSet}$ 
36: END FUNCTION
```
